Memoria
Sistemas Informáticos P2

Ejercicio 1:

Pasos realizados

Para crear el backend RPC se partió del proyecto base P1-base, que contiene la implementación inicial de la aplicación visaApp.

1. Copia del proyecto base: Se creó un nuevo directorio de trabajo denominado rpc-backend duplicando el contenido del proyecto original P1-base.

```
joseagm@DESKTOP-FV6C7JP:/mnt/c/Users/jose0/Desktop/SI2/SI2-Practica2$ ls
P1-base  P1-rpc-backend  rpc-backend  rpc-frontend
```

2. Renombrado de la aplicación: Para evitar conflictos y diferenciar claramente el rol de este componente, se renombró la aplicación de visaApp a visaAppRPCBackend. También, se realizó una sustitución global de todas las referencias al nombre antiguo en el código fuente mediante el comando sed.

```
joseagm@DESKTOP-FV6C7JP:/mnt/c/Users/jose0/Desktop/SI2/SI2-Practica2$ cd P1-rpc-backend/
joseagm@DESKTOP-FV6C7JP:/mnt/c/Users/jose0/Desktop/SI2/SI2-Practica2/P1-rpc-backend$ ls
README.md  build.sh  env  manage.py  requirements.txt  visaApp  visaSite
joseagm@DESKTOP-FV6C7JP:/mnt/c/Users/jose0/Desktop/SI2/SI2-Practica2/P1-rpc-backend$ mv visaApp visaAppRPCBackend
joseagm@DESKTOP-FV6C7JP:/mnt/c/Users/jose0/Desktop/SI2/SI2-Practica2/P1-rpc-backend$ ls
README.md  build.sh  env  manage.py  requirements.txt  visaAppRPCBackend  visaSite
joseagm@DESKTOP-FV6C7JP:/mnt/c/Users/jose0/Desktop/SI2/SI2-Practica2/P1-rpc-backend$ find ./ -type f -exec sed -i "s/visaApp/visaAppR
PCBackend/g" {} \;
```

3. Limpieza de componentes web: Dado que el backend RPC no requiere interfaz de usuario, se eliminaron todos los archivos y directorios destinados a la generación de HTML, tales como views.py, forms.py, la carpeta templates y los tests de vistas.

```
joseagm@DESKTOP-FV6C7JP:/mnt/c/Users/jose0/Desktop/SI2/SI2-Practica2/P1-rpc-backend$ rm visaAppRPCBackend/forms.py
rm visaAppRPCBackend/views.py
rm -rf visaAppRPCBackend/templates
rm visaAppRPCBackend/tests_views.py
rm visaAppRPCBackend/tests_models.py
joseagm@DESKTOP-FV6C7JP:/mnt/c/Users/jose0/Desktop/SI2/SI2-Practica2/P1-rpc-backend$ ls visaAppRPCBackend
__init__.py  admin.py  apps.py  management  migrations  models.py  pagoDB.py  urls.py
```

4. Configuración del entorno RPC:

- Se instaló la librería django-modern-rpc.
- Se modificó settings.py para incluir 'modernrpc' en INSTALLED_APPS.
- Se definió la variable MODERNRPC_METHODS_MODULES apuntando al fichero visaAppRPCBackend.pagoDB para registrar los métodos remotos.

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'visaAppRPCBackend.apps.AppConfig',
    'modernrpc',
]
```

```
MODERNRPC_METHODS_MODULES = [
    "visaAppRPCBackend.pagoDB"
]
```

Las plantillas no son necesarias porque el servidor RPC tiene una función puramente de servicio de datos estructurados (XML); su objetivo no es la representación visual (HTML) sino la comunicación entre procesos. Y los formularios se han eliminado debido a que los formularios de Django están diseñados para validar entradas manuales desde un navegador. En una arquitectura RPC, la responsabilidad de la validación inicial recae en el cliente, y el backend recibe los parámetros directamente para ser procesados por las funciones de Python.

Ejercicio 2:

Pasos realizados:

- Se ha editado el fichero pagoDB.py para importar el decorador @rpc_method de la librería modernrpc.
- Se ha aplicado dicho decorador a todas las funciones de interacción con la base de datos: verificar_tarjeta, registrar_pago, eliminar_pago y get_pagos_from_db, permitiendo su invocación remota mediante el protocolo XML-RPC.
- Se han modificado los métodos de retorno para transformar las instancias de modelos Django en diccionarios estándar de Python utilizando model_to_dict.

```python
from visaAppRPCBackend.models import Tarjeta, Pago
from modernrpc.core import rpc_method
from django.forms.models import model_to_dict


@rpc_method
def verificar_tarjeta(tarjeta_data):
    """ Check if the tarjeta is registered
    :param tarjeta_dict: dictionary with the tarjeta data
                         (as provided by TarjetaForm)
    :return True or False if tarjeta_data is not valid
    """
    if bool(tarjeta_data) is False or not Tarjeta.objects.filter(**tarjeta_data).exists():
        return False
    return True


@rpc_method
def registrar_pago(pago_dict):
    """ Register a payment in the database
    :param pago_dict: dictionary with the pago data (as provided by PagoForm)
      plus de tarjeta_id (numero) of the tarjeta
    :return new pago info if succesful, None otherwise
    """
    try:
        pago = Pago.objects.create(**pago_dict)
        # get default values from pago
        pago = Pago.objects.get(pk=pago.pk)
    except Exception as e:
        print("Error: Registrando pago: ", e)
        return None

    pago_a_devolver = model_to_dict(pago)
    pago_a_devolver['marcaTiempo'] = str(pago.marcaTiempo)

    return pago_a_devolver
```

```python
@rpc_method
def eliminar_pago(idPago):
    """ Delete a pago in the database
    :param idPago: id of the pago to be deleted
    :return True if succesful,
     False otherwise
     """
    try:
        pago = Pago.objects.get(id=idPago)
    except Pago.DoesNotExist:
        return False
    pago.delete()
    return True


@rpc_method
def get_pagos_from_db(idComercio):
    """ Gets pagos in the database corresponding to some idComercio
    :param idComercio: id of the comercio to get pagos from
    :return list of pagos found
     """
    pagos = Pago.objects.filter(idComercio=idComercio)

    lista_pagos = []

    for pago in pagos:
        pago_dict = model_to_dict(pago)
        pago_dict['marcaTiempo'] = str(pago.marcaTiempo)
        lista_pagos.append(pago_dict)

    return lista_pagos
```

```python
"""
URL configuration for PagoProj project.

The `urlpatterns` list routes URLs to views. For more information please see:
    https://docs.djangoproject.com/en/4.2/topics/http/urls/
Examples:
Function views
    1. Add an import:  from my_app import views
    2. Add a URL to urlpatterns:  path('', views.home, name='home')
Class-based views
    1. Add an import:  from other_app.views import Home
    2. Add a URL to urlpatterns:  path('', Home.as_view(), name='home')
Including another URLconf
    1. Import the include() function: from django.urls import include, path
    2. Add a URL to urlpatterns:  path('blog/', include('blog.urls'))
"""
from django.urls import path
from modernrpc.views import RPCEntryPoint

urlpatterns = [
    path("rpc/", RPCEntryPoint.as_view(), name="rpc")
]
```

Es fundamental utilizar model_to_dict porque los objetos de modelo de Django (como la instancia pago) son estructuras complejas que contienen metadatos de la base de datos y no pueden ser serializadas directamente a XML por el servidor RPC. Este método extrae los

datos y los convierte en un diccionario estandarizado que el protocolo XML-RPC sí puede procesar para su envío al cliente.

Adicionalmente, dado que model_to_dict solo procesa campos editables y omite automáticamente campos automáticos, ha sido necesario añadir manualmente el campo marcaTiempo convirtiéndolo a cadena de texto (str) para garantizar que el cliente reciba la información temporal completa del registro.

Ejercicio 3:

```
# IMPORTANT: this file should not be in a repository
# To remove a file named env from a Git repository
# but keep it in the source (local system), follow these steps:
# Remove the file from Git tracking but keep it locally
## git rm --cached env
# Add 'env' to .gitignore (so it's not tracked again)
## echo "env" >> .gitignore
# Commit the changes
## git commit -m "Removed env from Git tracking and added to .gitignore"
# Push the changes to the remote repository
## git push
# use sqlite 3
##DATABASE_SERVER_URL=sqlite:///db.sqlite3
# use postgres
DATABASE_SERVER_URL='postgres://alumnodb:alumnodb@localhost:15432/si2db'
# The client does not need to store data in any database
# so let us define a sqlite in orden to avoid warning messages
DEBUG=True
SECRET_KEY = 'django-insecure-alczftn)j1#$v%xmk@5j(n*px43c8kxgi_ua4%khc+t7g_)s9d'
```

Pasos realizados:

- Configuración de la VM1 como servidor de BD: Se arrancó la si2vm1 y se verificó la existencia de la base de datos si2db y el usuario alumnodb. Se habilitó el acceso SSH para gestión remota y se configuró PostgreSQL para escuchar en todas las interfaces (0.0.0.0).
- Configuración del entorno: Se definió la variable DATABASE_SERVER_URL en el archivo env apuntando a localhost:15432, puerto que VirtualBox redirige al 5432 de la VM.
- Ejecución de Migraciones: Se ejecutó python3 manage.py migrate para crear la estructura de tablas en el servidor remoto de la VM1.
- Población de datos: Se ejecutó el comando python3 manage.py populate para inicializar la base de datos con los registros de las tarjetas bancarias necesarios para las pruebas.

- Validación del punto de acceso: Se inició el servidor de desarrollo (runserver 8001) y se accedió a la URL del punto de entrada RPC. Se obtuvo la respuesta esperada por el sistema.

Ejercicio 4:

Ejercicio 5:

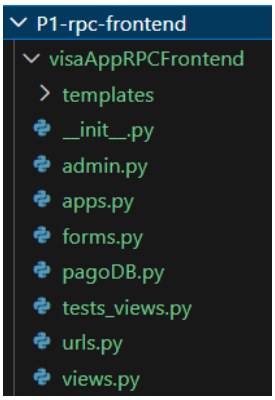
1. Se verifica que el directorio P1-rpc-frontend fue creado correctamente copiando el contenido de P1-base:

```
P1-base
P1-rpc-frontend
rpc-backend
rpc-frontend
.gitignore
PS C:\Users\rj9co\Desktop\Ingenieria Informática\3º de Informática\2º Cua
tri\SI II\Practicas\P2\SI2-Practica2>
```

2. La aplicación fue renombrada de *visaApp* a *visaAppRPCFrontend*:

```
\P2\SI2-Practica2\P1-rpc-frontend>
```

3. Se eliminaron los archivos innecesarios: *models.py, migrations, management, tests_models.py*



4. Se configuró la variable RPCAPIBASEURL en el archivo env con la URL del servidor RPC

```
# URL del servidor RPC Backend
RPCAPIBASEURL=http://localhost:28000/visaAppRPCBackend/rpc/
```

5. Se añadió la variable RPCAPIBASEURL en settings.py para leer el valor del archivo env

```
# URL del servidor RPC Backend
RPCAPIBASEURL = os.environ.get("RPCAPIBASEURL")
```

¿Por qué no se necesitan modelos en el cliente RPC?

No necesitamos modelos en el cliente porque el cliente no toca la base de datos directamente. El servidor es el que tiene los modelos y la BD. El cliente solo se conecta al servidor por RPC, le pide datos, el servidor los procesa y se los devuelve como diccionarios. El cliente solo los muestra en pantalla. Por eso no necesitamos models.py ni migraciones aquí.